MiniSpotify — Design Document

Key Changes to the Implementation

The original `LegacyMusicService` used `ArrayList`-based linear scans for nearly every lookup operation — songs, users, and playlists were all found by iterating from the start until a match was found. This made catalog loading itself O(n²) (each `addSong` call re-scanned the growing catalog to check for duplicates), and made exact artist/genre search O(n) per call regardless of catalog size. I replaced these linear scans with `Map`-based structures that give O(1) expected lookup, and built two additional indexes (by artist, by genre) so that exact-match search never needs to touch the full catalog.

Beyond performance, the inherited code had several encapsulation gaps: `Playlist.getSongs()` returned the live internal list directly, and `Playlist`'s mutator methods (`addSong`, `removeFirst`) were `public`, meaning any caller holding a `Playlist` reference (such as one returned from `getFollowedPlaylists`) could bypass `MusicService`'s permission checks entirely. These were fixed by making the mutators package-private and having all `MusicService` methods that return collections return either a copy or an unmodifiable view instead of the underlying mutable structure.

The four previously-unimplemented features (collaborators, following, favorites-as-duplicates, and recommendations) were built using `Set` semantics specifically because sets do not allow for duplicates automatically.

The most involved design decision was bounded listening history, which needed to satisfy three requirements simultaneously: capped at 50 entries, unique per song, and reordered to the most-recent position on replay. This was solved with a `LinkedHashMap` in access-order mode, with `removeEldestEntry` overridden to evict past 50 entries — a single data structure satisfying all three requirements without any manual bookkeeping.

Two bugs were found and fixed in the inherited code during this process: `Song`'s constructor used `Objects.requireNonNull(...)`, which throws `NullPointerException` rather than the `IllegalArgumentException` the spec requires for all invalid arguments; and `getTopSongs`'s tie-breaking comparator compared titles using `String`'s natural ordering, which is case-sensitive and would incorrectly sort a title like `"Banana"` before `"apple"` (since uppercase letters have lower Unicode values than lowercase ones).

Collection Choices

Field / Index	Declared Type	Concrete Collection	Reason for Choice	Expected Complexity
`catalog`	`Map<String, Song>`	`LinkedHashMap`	Needs O(1) lookup by song ID *and* insertion-order iteration. A plain `HashMap` would give O(1) lookup but no ordering guarantee; `LinkedHashMap` gives both at once.	`get`/`put`: O(1) expected. Full iteration (title search): O(n).
`artistIndex`	`Map<String, List<Song>>`	`HashMap<String, ArrayList<Song>>`	Exact artist search must avoid a full catalog scan. Keying by lowercased artist name groups all matching songs together for O(1) bucket access; the bucket itself is an `ArrayList` to preserve insertion order within that artist, with no uniqueness concern since a song is only ever added to one artist bucket.	`computeIfAbsent`/ lookup: O(1) expected. Returning k matches: O(k).
`genreIndex`	`Map<String, List<Song>>`	`HashMap<String, ArrayList<Song>>`	Same as `artistIndex`, applied to genre search. Also reused directly as the candidate source for `getRecommendations`.	Same as `artistIndex`.

`users`	`Map<String, User>`	`HashMap`	Usernames must be unique and are looked up by exact string match with no ordering requirement, making `HashMap` the simplest fit for O(1) lookup.	`get`/`put`: O(1) expected.
`playlists` (inside `User`)	`Map<String, Playlist>`	`HashMap`	Playlist names need only be unique within one owner, so each `User` holds its own independent map, naturally scoping uniqueness per spec.	`get`/`put`: O(1) expected.
`songs` (inside `Playlist`)	`List<Song>`	`ArrayList`	Playlist contents must preserve insertion order and explicitly allow duplicate songs — a `List` is the only Collections Framework type that allows both.	`addSong`: amortized O(1). `removeFirst` (first matching occurrence): O(n).
`collaborators` (inside `Playlist`)	`Set<String>`	`HashSet`	"Adding the same collaborator twice must not create duplicate permissions" is exactly a `Set`'s contract; no ordering is required.	`add`/`contains`: O(1) expected.

`followedPlaylists` (inside `User`)	`Set<Playlist>`	`HashSet`	Following the same playlist repeatedly must not be duplicated; each playlist is an object (created once in `createPlaylist`), so default reference equality correctly identifies "the same playlist" without needing to override `equals()`/`hashCode()`.	`add`/`contains`: O(1) expected.
`favorites` (inside `User`)	`Set<Song>`	`HashSet`	Favorites must be unique; `Song.equals()`/`hashCode()` are already defined by ID, making `HashSet` a direct fit with no extra duplicate-checking logic needed.	`add`/`remove`/`contains`: O(1) expected.
`history` (inside `User`)	`Map<String, Song>`	`LinkedHashMap` (access-order mode, `removeEldestEntry` overridden)	Must simultaneously be: bounded to 50 entries, unique per song, and reordered to most-recent on replay. Access-order mode moves an existing key to the end on re-`put`, and the overridden `removeEldestEntry` evicts past 50 — one structure satisfying all three requirements at once.	`put` (play/replay): O(1) expected. `getRecentlyPlayed` (reverse + truncate): O(k) for history size k ≤ 50.

`allTimePlayed` (inside `User`)	`Set<Song>`	`HashSet`	Recommendations must exclude every song ever played, not just the 50 retained in bounded history — this is tracked separately and unboundedly, with only membership testing needed, no ordering.	`add`/`contains`: O(1) expected.
`playCounts`	`Map<String,Integer>`	`HashMap`	Global play count per song, looked up by ID with no ordering requirement.	`merge`/`get`: O(1) expected.

Measured Performance

On a 100,000-song catalog: loading took ~210ms (down from the legacy implementation's documented ~minute-scale load, a direct consequence of fixing the O(n²) duplicate-check during load), 100,000 `getSong` calls completed in 5–10ms total, and 10,000 `searchByArtist` calls completed in 15–24ms total which is expected with O(1) expected behavior per operation, and comfortably within the spec's "well under 10 seconds" calibration bar even at one-fifth the stated 500,000-song target scale.